

逆向工程教學：從程式碼到遊戲規格書到自動化開發流程

適用對象：想要從一個「已完成的遊戲專案」出發，建立可複用的開發標準流程的開發者。**最終產出：**GDD（遊戲規格書）+ Agent Skills（職能方法論）+ Workflow（自動化撰寫流程）

本教學的經驗來源

這份教學文件並非憑空設計，而是從一次完整的逆向工程實戰中提煉而來。在實戰過程中，我們遇到了大量的陷阱、走過不少彎路，最終歸納出可重複使用的方法論。

以下是實戰中發現的**關鍵教訓**，已融入各階段的步驟和提示詞中：

階段	踩過的坑	修正後的做法
GDD 逆向	Texture 繪製順序沒記錄，導致視覺無法還原	每個程式生成 Texture 必須記錄繪製順序（由外到內 or 由內到外）
GDD 逆向	難度系統只寫「形狀種類」，實際上有兩個獨立參數軸	必須分析完整的配置結構，區分每個獨立維度
GDD 逆向	公式模糊帶過，寫「有震動」而非完整公式	所有公式必須完整記錄，包括每個變數的影響
GDD 逆向	環境氛圍元素（極淡背景、微格線）被忽略	「感覺不到但不能沒有」的元素也要規格化
Skills	第一版 Skills 充滿硬編碼數值（gravity=2.2），變成逆向工程文件	Skills 是方法論，用範例舉例，不強制數值
Skills	Critical Rules 寫成「必須設為某值」，限制了創意空間	改為品質標準：「必須記錄完整」，而非「必須等於某值」
Workflow	沒有 Sub-workflow，Phase 太粗糙，容易遺漏細節	每個 Phase 拆為獨立的 Sub-workflow，每步有檢查點

大多數遊戲專案在開發完成後，知識散落在程式碼的各個角落——常數定義、碰撞邏輯、渲染管線、UI 互動。這份教學帶你透過 **3 個階段**，將這些隱性知識系統化：

階段一	階段二	階段三
程式碼	遊戲規格書 (GDD)	Agent Skills
逆向工程	提煉方法論	組裝流程
Workflow		
(實作層)	(設計層)	(方法論層)
(自動化層)		

三個階段總覽

階段	輸入	輸出	核心動作
一、逆向推導 GDD	程式碼 (src/*.ts)	9 章 GDD 文件	讀碼→提取→結構化

二、推導 Agent Skills	GDD + 參考 Skills (temp/*.md)	7 個職能 Skill 檔案	去耦合→模板化→定品質標準
三、建立 Workflow	Agent Skills	主流程 + 子流程檔案	拆階段→加檢查點→串依賴

教學文件索引

#	檔案	內容
00	本文件 (00_overview.md)	總覽與三階段概述
01	01_reverse_engineer_gdd.md	階段一：程式碼 → GDD 的完整流程與提示詞
02	02_derive_agent_skills.md	階段二：GDD + 參考 → Agent Skills 的推導方法
03	03_create_workflows.md	階段三：建立 Workflow & Sub-workflows

專案結構參考

程式碼結構（需先掃描辨識）

典型的 Suika-type 遊戲專案，原始碼會涵蓋以下子系統：

原始碼目錄/		
├─ [入口檔案]	←	應用程式入口
├─ [遊戲主控制器]	←	遊戲生命週期管理
├─ [資料/配置]		
│ └─ [常數定義]	←	物理、形狀、難度等所有可調參數
│ └─ [型別定義]	←	介面/型別
├─ [核心系統]		
│ └─ [物理引擎整合]	←	Matter.js / Box2D 等
│ └─ [合成邏輯]	←	核心：碰撞→合成→得分
│ └─ [粒子系統]	←	合成粒子、環境粒子
│ └─ [輸入處理]	←	滑鼠/觸控→遊戲座標
├─ [渲染系統]		
│ └─ [圖層管理]	←	z-order, Container 結構
│ └─ [Texture 生成/快取]	←	程式繪製材質 + 快取策略
│ └─ [視窗縮放]	←	響應式縮放邏輯
├─ [音效系統]	←	Web Audio API / 素材播放
└─ [UI 系統]		
│ └─ [HUD]	←	分數、NEXT 預覽、按鈕
│ └─ [Overlay 彈窗]	←	難度選擇、Game Over、設定
│ └─ [排行榜]	←	線上/本地排行

📌 實際檔案名稱因專案而異，在階段一 Step 1 會透過掃描辨識出來。

參考 Skills (temp/*.md)

```
temp/
├─ game-designer.md          ← 通用遊戲系統設計師
├─ level-designer.md        ← 通用關卡設計師
├─ technical-artist.md      ← 通用技術美術師
├─ design-ui-designer.md    ← 通用 UI 設計師
├─ design-ux-architect.md   ← 通用 UX 架構師
├─ game-audio-engineer.md   ← 通用音效工程師
└─ narrative-designer.md    ← 通用敘事設計師
```

最終產出結構

```
docs/gdd/                      ← 遊戲規格書
├─ 00_index.md
├─ 01_game_overview.md
├─ ...
└─ 09_testing.md

.agents/skills/                 ← Agent Skills (職能方法論)
├─ game-designer/SKILL.md
├─ level-designer/SKILL.md
├─ technical-artist/SKILL.md
├─ art-style-director/SKILL.md
├─ ui-ux-designer/SKILL.md
├─ audio-engineer/SKILL.md
└─ qa-tester/SKILL.md

.agents/workflows/             ← 自動化 workflow
├─ write-gdd.md                ← 主流程
├─ gdd-phase1-core-design.md   ← 子流程
├─ gdd-phase2-level-design.md
├─ ...
└─ gdd-phase8-10-specs-test-verify.md
```

階段一：從程式碼逆向工程，推導出遊戲規格書（GDD）

輸入：一個未知的物理落下益智遊戲專案的原始碼 輸出：9 章 GDD 文件 (docs/gdd/01~09) 核心原則：不是「複製程式碼」，而是「還原設計意圖」—— 每段規格都要回答「是什麼」和「為什麼」

Step 1：掃描專案結構，建立模組地圖

目的

了解整個遊戲由哪些子系統組成，每個原始碼檔案負責什麼。

操作

- 1. 列出 src/ (或專案根目錄) 下所有原始碼檔案
- 2. 逐一讀取每個檔案的前 50 行 (import + 主要 export)
- 3. 整理成「模組職責表」和「依賴關係圖」

提示詞

請掃描此專案的原始碼目錄，列出所有原始碼檔案。
讀取每個檔案的 import 區塊和主要 export，
整理成一份表格：

檔案	子系統分類	主要職責	依賴哪些模組
-----	-----	-----	-----

子系統分類建議：

- 遊戲主控制器 (Game Controller)
- 資料/配置 (Data/Config)
- 物理引擎整合 (Physics)
- 合成/核心機制 (Core Mechanics)
- 粒子/視覺效果 (VFX/Particles)
- 渲染/圖層 (Rendering/Layers)
- 材質/Texture 生成 (Textures)
- 輸入處理 (Input)
- 物件池 (Object Pool)
- 音效 (Audio)
- UI / HUD
- 彈窗/Overlay
- 視窗縮放 (Viewport)

然後畫出一份 ASCII 的模組依賴圖。

預期產出

一份清楚的模組地圖，標明哪些檔案是「核心邏輯」、哪些是「渲染」、哪些是「UI」。這份地圖將作為後續所有分析的導航索引。

Step 2：提取核心常數與配置表

目的

找出遊戲的所有「可調參數」—— 這些是 GDD 數值表的來源。

操作

1. 從 Step 1 的模組表中，找到「資料/配置」分類的檔案
2. 如果找不到獨立配置檔，搜尋 `const`, `export const`, `enum` 等關鍵字
3. 分類提取所有常數
4. 對每個數值推測「設計意圖」

提示詞

在原始碼中搜尋遊戲的核心配置。
可能集中在一個 `config` 檔案，也可能分散在各模組。

搜尋策略：

1. 先找是否有獨立的配置/常數檔案（搜尋 `config`, `constants`, `settings` 等關鍵字）
2. 如果是分散式，搜尋 `export const` 找出所有常數定義

找到後，將所有常數分類整理為：

1. ****物理引擎參數****（重力、摩擦力、彈性、密度、迭代次數...）
2. ****形狀定義表****（每個等級的半徑、顏色、邊數、得分）
3. ****難度配置****（各難度改了哪些參數、有幾種難度）
4. ****空間規格****（設計解析度、遊戲區域大小、邊距、判定線位置）
5. ****遊戲機制參數****（COMBO 窗口、投放冷卻、碰撞冷卻...）
6. ****UI/視覺參數****（字型大小、動畫時長、`z-index`...）
7. ****音效參數****（頻率公式、音量範圍...）

每個數值旁邊，請推測其「設計意圖」——
例如：如果重力值比預設值（`1.0`）大，
設計意圖可能是「讓形狀快速落地，節奏緊湊」。

品質檢查

- ☐ 所有 `export const` 都已收錄
- ☐ 每個數值有分類
- ☐ 有設計意圖推測

Step 3：解構核心機制（合成邏輯）

目的

還原遊戲最核心的機制 —— 形狀碰撞合成的完整流程。

操作

1. 從 Step 1 的模組表中，找到「核心機制/合成」分類的檔案
2. 搜尋碰撞事件相關代碼（如 `collisionStart`, `onCollision`, `collision` 等關鍵字）
3. 追蹤從「碰撞事件」到「合成完成」的每一步
4. 記錄所有數值公式

提示詞

請在原始碼中找到處理「形狀合成」的邏輯。

搜尋策略：

1. 搜尋 `collision`, `merge`, `combine`, `fuse` 等關鍵字
2. 找到碰撞事件的監聽器（通常掛在物理引擎的 `collision` 事件上）
3. 追蹤碰撞後的處理鏈

找到後，請逐步描述完整的處理流程，
使用以下格式記錄：

機制：形狀合成

****觸發條件****：[怎麼判斷兩個形狀可以合成？比對什麼屬性？]

****處理流程****：

1. [標記/移除舊形狀]
2. [計算合成位置（中點？加權？）]
3. [生成新形狀]
4. [新形狀是否有初始速度？ v_x/v_y 怎麼算？]
5. [計算得分]
6. [COMBO 判定與更新]
7. [觸發粒子效果]
8. [觸發音效]
9. [觸發畫面震動]

****公式記錄****：

- 合成位置公式：[...]
- 得分公式：[...]
- 震動強度公式：[...]
- COMBO 窗口時間：[...]

****邊界條件****：

- 最高等級碰撞時：[...]

- 合成冷卻期：[...]
- 多對同時碰撞：[...]

Step 4：解構渲染與視覺系統

目的

還原遊戲的視覺層次結構、Texture 生成邏輯、以及視覺效果的精確參數。

操作

1. 從模組表中找到「渲染/圖層」和「Texture」相關檔案
2. 分析圖層架構
3. 分析 Texture 的程式生成方式
4. 分析粒子效果規格

提示詞（圖層）

請在原始碼中找到管理渲染圖層（Layers / Containers）的邏輯。

搜尋策略：

1. 搜尋 Container, layer, addChild, zIndex 等關鍵字
2. 找到建立圖層結構的代碼

列出所有渲染圖層，從底到頂：

#	變數名	類型	用途	特殊設定
---	-----	-----	-----	-----

提示詞（Texture 生成）

請在原始碼中找到 Texture 的生成與快取邏輯。

搜尋策略：

1. 搜尋 Graphics, Texture, generateTexture, drawCircle, drawRect 等關鍵字
2. 找到所有用程式碼繪製材質的函數

對每個 Texture 生成函數，描述：

1. 繪製了幾層？繪製順序（由外到內 or 由內到外）？
2. 每層的半徑/尺寸、顏色、alpha 值
3. 快取策略（Key 格式、何時清除）

品質要求：規格必須足夠精確，讓另一個開發者能從此描述「完全還原」出相同的視覺效果。

提示詞（粒子系統）

請在原始碼中找到粒子系統的實作。

搜尋策略：

1. 搜尋 `particle`, `emit`, `spawn`, `life`, `decay` 等關鍵字
2. 找到粒子的資料結構定義和更新邏輯

列出所有粒子類型及其完整規格：

粒子類型	數量公式	速度範圍	衰減率	重力影響	使用的 Texture
-----	-----	-----	-----	-----	-----

額外記錄：

- 生命週期管理方式
- 是否使用 `Object Pool`？
- 效能保護機制（最大同時數量）

Step 5：解構 UI/UX 系統

目的

還原 HUD 版面、Overlay 彈窗系統、互動狀態、視窗縮放策略。

操作

1. 從模組表中找到「UI/HUD」和「Overlay」相關檔案
2. 分析 HUD 元素的渲染方式（是 HTML DOM 還是 Canvas）
3. 分析 Overlay 的開關邏輯和前景狀態管理
4. 分析視窗縮放公式

提示詞

請在原始碼中分析 UI 系統。

搜尋策略：

1. 搜尋 `overlay`, `modal`, `popup`, `hud`, `score`, `getElementById`, `createElement` 等關鍵字
2. 區分哪些 UI 是 HTML DOM、哪些是 Canvas 繪製

回答以下問題：

****HUD（常駐介面）**：**

1. 有哪些 HUD 元素？（分數、NEXT 預覽、功能按鈕...）
2. 每個元素的渲染方式（HTML 或 Canvas）？
3. 字型、字號、顏色規格？

****Overlay (彈窗) **：**

1. 有哪些 Overlay？它們的 z-index 層級？
2. 開啟一個 Overlay 時，其他 Overlay 怎麼處理？
3. 有沒有「前景狀態保存/恢復」機制？

****視窗縮放**：**

1. 縮放公式？（搜尋 scale, transform, resize）
2. transformOrigin 設在哪？
3. 有無 GPU 加速設定？

****Design Token**：**

1. 有沒有 CSS 變數系統（搜尋 :root, var(--）？
2. 有沒有獨立的設計系統檔案？

Step 6：解構音效系統

提示詞

請在原始碼中分析音效系統。

搜尋策略：

1. 搜尋 Audio, AudioContext, oscillator, gain, playSound, sfx 等關鍵字
2. 判斷音效是「預錄素材」還是「程式即時合成」

回答以下問題：

1. 音效實作方式：Web Audio API 程式合成？還是音檔播放？
2. 如果是程式合成：
 - 合成音效有幾個波形圖層？
 - 每層的振盪器類型 (sine, square, triangle...) 和頻率公式？
 - 等級 (level) 如何影響音高？
3. 有沒有音效包 (Sound Pack) 切換機制？
4. 最大同時發聲數限制？
5. 靜音/音量控制邏輯？

Step 7：解構物理引擎整合

提示詞

請在原始碼中分析物理引擎的整合方式。

搜尋策略：

1. 搜尋 Engine, World, Body, Matter, Box2D,

gravity, restitution, friction 等關鍵字

2. 找到引擎初始化和 Body 建立的代碼

回答以下問題：

****引擎選型****：使用什麼物理引擎？版本？

****引擎全域參數****（完整列出，不可遺漏）：

參數	值	影響
gravity.x	?	...
gravity.y	?	...
...iterations	?	...

****形狀 Body 參數****：

參數	值	影響
restitution	?	碰撞彈性
friction	?	滑動摩擦
density	?	或公式 質量感

****牆壁 Body 參數****（如果與形狀不同，單獨列表）

****碰撞體設計****：

- 碰撞體形狀（圓形？多邊形？）
- 碰撞半徑公式（視覺半徑 = 碰撞半徑？還是有差異？）

****Game Over 判定****：

- 判定公式在哪裡？
- 判定條件是什麼（靜止後超過某條線？）
- 有沒有防誤判機制？

Step 8：解構遊戲流程與狀態管理

提示詞

請在原始碼中分析遊戲的完整生命週期。

搜尋策略：

1. 找到遊戲主控制器（通常是最大的檔案、或入口引用的第一個模組）
2. 搜尋 init, start, update, tick, gameOver, reset, restart 等關鍵字

回答以下問題：

****初始化流程**：**

遊戲啟動時，依序初始化了哪些子系統？順序是什麼？

****主循環（每幀更新）**：**

每幀（Ticker / requestAnimationFrame）做了哪些事？

順序是什麼？

****Game Over 流程**：**

1. 判定觸發後做了什麼？

2. 有沒有延遲/動畫？

3. 是否上傳分數？

4. 顯示什麼 UI？

****重新開始/重置流程**：**

清理了哪些東西？（Body, Sprite, 粒子, 分數, COMBO...）

清理的順序重要嗎？有沒有殘留的風險？

****難度切換**：**

切換時做了什麼特別的事？

Texture 快取是否重建？

Step 9：組裝成 GDD

目的

將 Step 1-8 的分析結果，組裝成結構化的 GDD 文件。

提示詞

請根據以上所有分析結果（Step 1~8），

將它們組裝為結構化的 GDD。

產出以下 9 個章節檔案，放在 docs/gdd/ 目錄下：

#	檔案名	內容來源
01	01_game_overview.md	Step 1 架構 + Step 3 核心機制 + Step 2 難度
02	02_scene_and_level_design.md	Step 2 空間+形狀表 + Step 4 圖層
03	03_technical_foundation.md	Step 7 物理 + Step 4 渲染 + Step 2 所有常數
04	04_art_style_and_narrative.md	Step 4 Texture + 視覺效果
05	05_ui_ux_design.md	Step 5 全部
06	06_game_flow.md	Step 8 全部
07	07_audio_design.md	Step 6 全部
08	08_additional_specs.md	Step 4 粒子 + Texture 繪製規格

品質要求：

1. 數值精確——與原始碼一致，標註常數名稱以便追溯
2. 設計意圖透明——每個「是什麼」都附帶「為什麼（推測）」
3. 公式完整——不可只寫「有震動效果」，要寫出算式
4. 跨章節一致——同一數值在不同章節的描述必須相同

品質檢查

- ☐ 9 個檔案全部產出
- ☐ 跨章節數值一致
- ☐ 物理參數完整（零遺漏）
- ☐ 每個機制有「觸發→處理→輸出→邊界條件」
- ☐ Texture 規格可還原

階段一完成標準

✅ 9 章 GDD 文件就緒於 docs/gdd/ 目錄下 ✅ 所有數值與原始碼常數一致 ✅ 所有公式完整記錄 ✅
跨章節無矛盾 ✅ 每個視覺效果規格可被還原

⚠️ 常見陷阱（實戰經驗）

以下是實際逆向工程過程中踩過的坑——每一條都曾導致 GDD 與程式碼不一致：

陷阱 1：Texture 繪製順序沒記錄

問題：只記錄了 Glow 效果的「4 層同心圓」，但沒記錄繪製順序。由外到內畫和由內到外畫，最終視覺結果完全不同（內層覆蓋外層 vs 外層覆蓋內層）。

解法：每個程式生成的 Texture，必須記錄：

- 繪製順序（由外到內 or 由內到外）
- 每層的精確 alpha 值（不可寫「漸變」，要寫 `0.08 / i`）

陷阱 2：難度系統被簡化為單一維度

問題：GDD 寫「Easy=前4種, Normal=前5種, Hard=前6種」，但實際上程式碼有兩個獨立的難度軸：形狀縮減（shapesSlice）和投放等級上限（maxDropLevel）。

解法：分析難度配置時，必須解構完整的配置資料結構，把每個獨立的參數軸都列出來，不可籠統概括。

陷阱 3：公式模糊帶過

問題：GDD 寫「合成時有畫面震動效果」，但沒記錄公式 `4 + level × 1.5 + comboCount × 0.5`。

解法：搜尋所有包含數學運算的代碼行，特別是涉及 level 或 combo 的公式，全部精確記錄。

陷阱 4：環境氛圍元素被忽略

問題：微格線 (alpha=0.025)、微弱背景光等「感覺不到但不能沒有」的元素被忽略。移除後遊戲氛圍明顯下降。

解法：分析渲染代碼時，注意所有 `alpha < 0.1` 的繪製。這些通常是環境氛圍元素，雖然不顯眼但對「空間感」至關重要。

陷阱 5：Overlay 狀態保存機制遺漏

問題：GDD 寫了每個 Overlay 的內容，但沒記錄它們之間的互動：開啟子面板時需要記住、隱藏、並在關閉後恢復父面板。

解法：分析 Overlay 時，不只看每個面板「長什麼樣」，還要追蹤狀態管理邏輯——特別是 `show()` , `hide()` , `save/restore` 的呼叫鏈。

陷阱 6：Viewport 縮放描述不精確

問題：GDD 寫「等比縮放」，但實際上是「純高度對齊」(`scale = viewH / naturalH`)，兩者行為不同。

解法：找到 `resize` 處理函數，逐行分析 `scale` 的計算公式，不可用「等比縮放」等模糊詞帶過。

陷阱 7：物理參數遺漏

問題：只記錄了「重要的」物理參數（重力、彈性），遺漏了 `frictionStatic` , `frictionAir` , `baseDensity` , `densityPerLevel` 等。

解法：在物理引擎初始化和 Body 建立的代碼中，用搜尋的方式找出所有被設定的屬性，不可只靠肉眼挑選。

陷阱 8：合成後的新形狀物理行為沒記錄

問題：記錄了「兩個形狀合成為新形狀」，但沒記錄新形狀的初始速度 (`vx=±0.75`, `vy=-1.5`) —— 這個微小的上拋效果是「彈跳感」的來源。

解法：合成邏輯分析時，除了追蹤「消除→生成」，還要追蹤生成後的所有 `Body.setVelocity` 或 `applyForce` 呼叫。

階段二：從 GDD + 參考 Skills，推導出專屬 Agent Skills

輸入：

- 階段一產出的 GDD (docs/gdd/01~09)
- 通用遊戲開發 Agent Skills 參考 (temp/*.md)

輸出：7 個專屬於此遊戲類型的 Agent Skill 檔案 (.agents/skills/*/SKILL.md)

核心原則：Skills ≠ GDD 的副本。Skills 是方法論（怎麼寫好 GDD），不是內容（GDD 寫了什麼）

重要觀念：Skills 與 GDD 的區別

	GDD（規格書）	Skill（方法論）
內容	具體數值、公式、配置	撰寫模板、品質標準、設計原則
目的	讓工程師能實作	讓 AI 能寫出好的 GDD
舉例	「gravity.y = 2.2」	「物理參數必須完整列出，並附帶對遊戲手感的影響說明」
數值	硬編碼	用範例舉例，不強制數值

Step 1：分析參考 Skills 的結構

目的

理解通用 Agent Skills 的共同結構模式，作為自己 Skills 的骨架。

操作

讀取 temp/ 目錄下所有 7 個 .md 檔案，
分析它們的共同結構

提示詞

請讀取 temp/ 目錄下所有 .md 檔案（共 7 個），
分析它們的共同結構模式：

1. YAML frontmatter 包含哪些欄位？
2. 文件的章節結構是什麼？（通常有：Identity, Core Mission, Critical Rules, Workflows...）
3. Critical Rules 的寫法特色是什麼？
4. 每個 Skill 如何區分「必須做」和「建議做」？

整理成一份「Skill 結構模板」，
我將用這個模板來撰寫自己的 Skills。

預期產出

Skill 結構模板

name: [名稱]

description: [一句話描述職能]

【職能全名】

職能 : [做什麼]

When to Use

— [觸發場景列表]

品質標準

【標準類別】

— [具體規則 + 範例]

【撰寫指南/模板】

【子主題】

[空白模板 + 範例]

工作流程

1. ...

Step 2：職能分組——決定需要哪些 Skills

目的

根據 GDD 的章節分類 + 參考 Skills 的職能覆蓋，決定最終需要的 Skill 數量與分工。

操作

1. 列出 GDD 的 9 個章節
2. 列出 temp/ 的 7 個參考 Skill
3. 建立「GDD 章節 × Skill 職能」的對照矩陣
4. 決定需要合併、拆分或新增哪些職能

提示詞

根據我的 GDD 9 個章節，和 temp/ 的 7 個參考 Skills，
請建立一份對照矩陣：

| GDD 章節 | 對應的參考 Skill | 需要的專屬 Skill | 差異/調整 |

|-----|-----|-----|-----|

然後回答：

1. 哪些參考 Skill 可以直接沿用？
2. 哪些需要合併？（例如 ui-designer + ux-architect → ui-ux-designer）
3. 哪些需要新增？（例如參考中沒有 QA，但 GDD 需要測試章節）
4. narrative-designer 在這類遊戲中扮演什麼角色？需要獨立還是併入 art-style-director？

預期產出——職能決策表

最終決定的 7 個 Skills：

- | | |
|-----------------------|--|
| 1. game-designer | ← 對應 temp/game-designer.md |
| 2. level-designer | ← 對應 temp/level-designer.md |
| 3. technical-artist | ← 對應 temp/technical-artist.md |
| 4. art-style-director | ← 合併 temp/narrative-designer.md 的敘事部分 |
| 5. ui-ux-designer | ← 合併 temp/design-ui-designer.md + design-ux-architect.md |
| 6. audio-engineer | ← 對應 temp/game-audio-engineer.md |
| 7. qa-tester | ← 新增（參考中無對應，但 GDD 第 9 章需要） |

Step 3：逐一撰寫每個 Skill

目的

以參考 Skills 為骨架，以 GDD 為內容來源，抽象出方法論。

核心轉換原則

GDD 說：「gravity.y = 2.2」

→ Skill 說：「物理參數必須完整列出，並附帶對遊戲手感的影響說明」

→ Skill 範例：「例如 gravity.y = 2.2，設計意圖：比真實重力快，讓落地節奏緊湊」

GDD 說：「Glow 層 alpha = 0.08/i，由外到內 4 層」

→ Skill 說：「Texture 規格必須記錄每層的尺寸/顏色/alpha/繪製順序，確保可還原」

→ Skill 範例：「例如 Glow 層 alpha = 0.08/i，由外到內繪製」

提示詞（以 game-designer 為例）

請幫我撰寫 .agents/skills/game-designer/SKILL.md。

****參考**：**

- 結構骨架：temp/game-designer.md（取其「Identity / Critical Rules / Workflow」的結構精神）
- 內容來源：docs/gdd/01_game_overview.md（取其設計支柱、核心循環、難度平衡的「議題」）

****要求**：**

1. YAML frontmatter: name, description (用中文)
2. 「品質標準」取代「Critical Rules」—— 定義撰寫 GDD 時的品質把關，而非程式碼規範
3. 提供「撰寫模板」而非具體數值——
例如：提供空白的「難度平衡表模板」，附帶填寫原則，而非填好的表
4. 硬數值只在「範例」中出現——
例如：「範例：gravity.y = 2.2，意圖：快速落地」
而非：「gravity.y 必須設為 2.2」
5. 工作流程用中文的步驟式描述

****重要**：**

- 不要照抄 GDD 的數值
- 不要限制創意——同一個模板可以填入不同的遊戲設定
- 聚焦「怎麼寫好 GDD」而非「GDD 寫了什麼」

其他 6 個 Skill 的提示詞模式

對每個 Skill，使用同樣的結構，只替換對應的參考和內容來源：

請幫我撰寫 `.agents/skills/[SKILL_NAME]/SKILL.md`。

****參考**：**

- 結構骨架：`temp/[REFERENCE_FILE].md`
- 內容來源：`docs/gdd/[GDD_CHAPTER].md`

****要求**：**

〔同上 5 條通用要求〕

****此 Skill 特別注意**：**

- 〔該職能特有的注意事項〕

各 Skill 的「特別注意」

Skill	特別注意
level-designer	空間規格需要 ASCII 圖模板；進化表需完整欄位定義
technical-artist	物理參數需分組模板（引擎/形狀/牆壁）；Texture 需可還原性標準
art-style-director	每個視覺元素需附帶「氛圍意圖」模板；Merge narrative-designer 的敘事部分
ui-ux-designer	合併 UI + UX 兩個參考；需定義雙層渲染（DOM/Canvas）決策原則
audio-engineer	程式合成音效的圖層模板；Voice Budget 分桌機/行動
qa-tester	無直接參考——從 GDD 09 測試章節抽象出測試方法論

Step 4：品質檢查——驗證 Skills 的方法論純度

提示詞

請檢查剛撰寫的 7 個 Skill 檔案，
驗證以下 3 項標準：

1. ****無硬編碼數值****：
搜尋所有出現的數字（如 2.2, 0.08, 1200），
判斷是「範例中的參考值」還是「強制的規定值」。
如果是後者，改寫為範例形式。
2. ****有撰寫模板****：
每個 Skill 至少提供 2 個「空白可填寫的模板」。
檢查模板是否足夠通用（可用於不同風格的同類遊戲）。
3. ****有品質標準****：
每個 Skill 至少定義 3 條品質標準。
品質標準應該是可檢查的（不是「寫得好」，而是「每個數值附帶設計意圖」）。

列出所有不合格的地方，並提供修改建議。

階段二完成標準

✅ 7 個 Skill 檔案就緒於 `.agents/skills/*/SKILL.md` ✅ 無硬編碼數值（只在範例中出現） ✅ 每個 Skill 有撰寫模板 + 品質標準 ✅ Skills 聚焦方法論，不限制創意

⚠️ 常見陷阱（實戰經驗）

陷阱 1：Skill 變成 GDD 的副本

問題：第一版 Skills 寫了 `gravity.y = 2.2`、`friction = 0.6` 這類硬編碼數值。這讓 Skill 變成了「GDD 的逆向工程參考文件」，而非「方法論」。

解法：嚴格區分：

- ❌ Skill 寫：「`gravity.y` 必須設為 2.2」
- ✅ Skill 寫：「物理參數必須完整列出，並附帶手感影響說明」
範例：「`gravity.y = 2.2`，意圖：快速落地，節奏緊湊」

陷阱 2：Critical Rules 限制了創意

問題：第一版的 Critical Rules 寫成「GlowTexture 必須是 4 層同心圓，半徑 32/22/14/8px」。這等於鎖死了設計空間，未來換風格就得改 Skill。

解法：Critical Rules 改為「品質標準」——規定必須記錄什麼，而非必須等於什麼：

- ✗ 「Glow 層 alpha 必須是 $0.08/i$ 」
- ✓ 「Texture 規格必須記錄每層的 alpha 公式和繪製順序，確保可還原」

陷阱 3：忘記合併或拆分參考 Skills

問題：參考 temp/ 中有 design-ui-designer.md 和 design-ux-architect.md 兩個檔案，但在此類遊戲中 UI 和 UX 的分工界線很模糊。同時沒有 QA Skill，但 GDD 需要測試章節。

解法：Step 2 的職能分組決策非常重要——不是照搬參考結構，而是根據自己 GDD 的實際章節分類來決定。

陷阱 4：品質標準不可量化

問題：品質標準寫「設計得好」、「視覺有質感」——這無法被 AI 執行。

解法：品質標準必須是可檢查的：

- ✗ 「視覺效果要好」
- ✓ 「每個視覺元素必須附帶氛圍意圖（這個元素對玩家產生什麼感受？）」
- ✓ 「每個 UI 元素必須標明渲染層級（HTML DOM / Canvas / 混合）」

階段三：建立 Workflow & Sub-workflows，自動化撰寫 GDD

輸入：階段二產出的 7 個 Agent Skills 輸出：1 個主工作流 + 6 個子工作流檔案 核心原則：Workflow 是「執行腳本」—— AI 照著走，就能產出高品質 GDD

為什麼需要 Workflow？

Skills 定義了「怎麼寫好 GDD」，但沒有定義「先寫什麼、後寫什麼、寫完怎麼驗收」。Workflow 做的就是：

- 1. 定義順序 — Phase 之間有依賴（先有形狀表才能寫配色）
- 2. 調用 Skills — 每個 Phase 明確使用哪個 Skill 的模板和品質標準
- 3. 設置檢查點 — 每步完成後確認品質
- 4. 交叉驗證 — 最終檢查跨章節一致性

Step 1：拆解 GDD 章節為有依賴關係的 Phase

操作

- 1. 列出 9 個 GDD 章節
- 2. 分析章節間的依賴關係（需要先有什麼才能寫什麼）
- 3. 合併或拆分，形成可執行的 Phase
- 4. 建立檢查點

提示詞

我的 GDD 有 9 個章節，Agent Skills 有 7 個。
請分析章節之間的依賴關係，並將其拆分為有前後順序的 Phase。

規則：

- 1. 每個 Phase 對應 1-2 個 GDD 章節
- 2. 每個 Phase 標明「要呼叫哪個 Agent Skill」
- 3. 標明前置依賴（需要先完成哪個 Phase）
- 4. 最後一個 Phase 做「交叉驗證」

請用以下格式呈現：

Phase	名稱	章節	Skill	前置依賴
-----	-----	-----	-----	-----

預期產出

Phase 1 核心設計	→ 01 章	→ game-designer	→ 無
Phase 2 空間設計	→ 02 章	→ level-designer	→ Phase 1
Phase 3 技術架構	→ 03 章	→ technical-artist	→ Phase 2
Phase 4 視覺風格	→ 04 章	→ art-style-director	→ Phase 2, 3
Phase 5 UI/UX	→ 05 章	→ ui-ux-designer	→ Phase 4
Phase 6 遊戲流程	→ 06 章	→ game+level	→ Phase 1-5
Phase 7 音效	→ 07 章	→ audio-engineer	→ Phase 1
Phase 8 特效規格	→ 08 章	→ tech+art	→ Phase 4
Phase 9 測試	→ 09 章	→ qa-tester	→ Phase 1-8
Phase 10 交叉驗證	→ 00 索引	→ 全部	→ Phase 1-9

Step 2：建立主工作流

操作

建立 `.agents/workflows/write-gdd.md`，作為統籌所有 Phase 的入口。

提示詞

請建立一個主工作流檔案 `.agents/workflows/write-gdd.md`。

格式要求：

1. YAML frontmatter 包含 description
2. 「前置準備」章節：
 - 確認遊戲定位的問題清單
 - 需要讀取的 Agent Skills 清單
 - GDD 目錄結構
3. 「執行流程」章節：
 - 逐一列出每個 Phase
 - 每個 Phase 標明：調用的 Skill、產出檔案、Steps 數量
 - 每個 Phase 附帶「→ 執行 sub-workflow: /gdd-phaseN-xxx」的引用
4. 「驗收標準」章節：
 - 5 項最終驗收標準

主流程不需要寫各 Phase 的細節——細節在 sub-workflow 中。

主流程的角色是「導航地圖」。

Step 3：建立 Sub-workflows

操作

為每個 Phase（或 Phase 組合）建立獨立的子工作流檔案。

提示詞模板

對每個 Sub-workflow，使用以下提示詞結構：

請建立子工作流 `.agents/workflows/gdd-phase [N] - [name].md`。

****Phase 資訊**：**

- 調用 Skill：[skill_name]
- 產出 GDD 章節：docs/gdd/[chapter].md
- 前置依賴：Phase [X] 完成

****要求**：**

1. YAML frontmatter 包含 description
2. 開頭標明調用的 Skill、產出檔案、前置依賴
3. 將此 Phase 拆分為 [N] 個 Steps

****每個 Step 必須包含**：**

- a) Step 標題和目的
- b) 具體的操作指引（要產出什麼格式的內容）
- c) 如果涉及模板，引用對應 Skill 中的模板
- d) 結尾有 3-5 條「檢查點」Checklist
（用 `- [ ]` 格式，讓 AI 可以逐條驗證）

****Phase 結尾必須包含**：**

- 「Phase N 完成驗收」清單
- 「→ 產出確認後進入 Phase N+1」的導航

各 Phase 的 Steps 參考

在呼叫生成子工作流時，可附帶以下參考提示，讓 AI 知道切分粒度：

Phase	建議的 Steps	說明
1 核心設計	設計支柱 → 核心循環 → 玩家輪廓 → 情感弧線 → 合成規格 → 難度平衡 → 得分經濟	7 Steps，最基礎的設計決策
2 空間設計	設計解析度 → 牆壁 → 圖層 → 難度精確規格 → 進化系統 → 數值分析	6 Steps，含 ASCII 空間圖
3 技術架構	架構圖 → 物理引擎 → 渲染初始化 → Texture 快取 → Pool → 輸入 → 持久化 → 建構	8 Steps，技術面最細
4 視覺風格	風格定義 → 配色 → 形狀繪製 → 環境場景 → 世界觀	5 Steps，含氛圍意圖
5 UI/UX	UI架構 → Token → HUD → Overlay → 回饋 → Viewport → 無障礙	7 Steps，含版面圖
6+7 流程+音效	初始化 → 主循環 → 合成流程 → GameOver → 重置 Sonic Identity → 音效 → 音效包 → Budget	9 Steps，兩個 Skill 合在一個檔案

8+9+10 特效 +測試+驗證	粒子 → 環境特效 → 效能預算 玩法測試 → 難度測試 → UI 測試 → 效能 → 跨平台 → 邊界 → GDD一致性 索引 → 一 致性驗證 → 最終驗收	13 Steps
---------------------	--	----------

Step 4：加入 AI 執行時的 Skill 調用指引

目的

確保 AI 在執行 Workflow 時，會去讀取對應的 Skill，並使用其中的模板和品質標準。

提示詞

請檢查所有 sub-workflow 檔案，
確認每個 Phase 的開頭都有以下格式：

> **調用 Skill**：`[skill-name]`
>
> **使用的模板**：
> - [Skill 中的模板 A 名稱]
> - [Skill 中的模板 B 名稱]
>
> **品質標準**（來自 Skill）：
> - [Skill 中的品質標準 1]
> - [Skill 中的品質標準 2]

如果某個 sub-workflow 缺少這些資訊，請補上。

Step 5：驗證 Workflow 的完整性

提示詞

請驗證 Workflow 系統的完整性：

1. **覆蓋率**：

- 9 個 GDD 章節是否都被某個 Phase 覆蓋？
- 7 個 Skills 是否都被某個 Phase 調用？

2. **依賴正確性**：

- 每個 Phase 的前置依賴是否合理？
- 是否有循環依賴？

3. **檢查點密度**：

- 每個 Step 是否有 3-5 條檢查點？
- 檢查點是否可量化（不是「寫得好」，而是「有 ASCII 圖」）？

4. **Slash Command 可用性**：

- 確認主流程可用 `/write-gdd` 啟動
- 確認每個子流程可單獨用 `/gdd-phaseN-xxx` 啟動

列出所有不合格項目和修正建議。

最終產出結構

```
.agents/workflows/  
├─ write-gdd.md                ← /write-gdd (主入口)  
├─ gdd-phase1-core-design.md   ← /gdd-phase1-core-design  
├─ gdd-phase2-level-design.md  ← /gdd-phase2-level-design  
├─ gdd-phase3-technical.md     ← /gdd-phase3-technical  
├─ gdd-phase4-art-style.md     ← /gdd-phase4-art-style  
├─ gdd-phase5-ui-ux.md         ← /gdd-phase5-ui-ux  
├─ gdd-phase6-7-flow-audio.md  ← /gdd-phase6-7-flow-audio  
└─ gdd-phase8-10-specs-test-verify.md ← /gdd-phase8-10-specs-test-verify
```

使用方式

```
# 完整流程  
/write-gdd  
  
# 只執行某個階段  
/gdd-phase1-core-design  
/gdd-phase4-art-style  
  
# 補寫特定章節  
/gdd-phase8-10-specs-test-verify
```

階段三完成標準

✅ 主工作流可用 `/write-gdd` 啟動 ✅ 每個子流程可單獨啟動 ✅ 每個 Step 有操作指引 + 檢查點 ✅ 每個 Phase 有完成驗收清單 ✅ Workflow 覆蓋全部 9 章 GDD 和 7 個 Skills ✅ 無循環依賴

⚠️ 常見陷阱（實戰經驗）

陷阱 1：Phase 粒度太粗，遺漏細節

問題：第一版 Workflow 只有 10 行的高層描述，執行時容易跳過「環境場景氛圍意圖」、「Overlay 狀態保存」這類細節。

解法：每個 Phase 拆為獨立的 Sub-workflow，每個 Sub-workflow 有 3-8 個 Steps，每步有 3-5 條 Checklist。具體到「這一步必須產出一個表格」的粒度。

陷阱 2：沒有跨 Phase 驗證

問題：Phase 2 寫了形狀進化表，Phase 4 寫了形狀配色表——兩邊的等級數、顏色可能不一致。

解法：Phase 10（交叉驗證）做三維一致性檢查：

1. 數值一致性 — 同一數值跨章節相同
2. 命名一致性 — 常數名、圖層名跨章節相同
3. 邏輯一致性 — 流程中提到的系統都在架構圖中

陷阱 3：Sub-workflow 沒有引用 Skill

問題：Sub-workflow 寫了步驟，但沒標明「使用哪個 Skill 的哪個模板」。AI 執行時不知道去讀 Skill，產出品質下降。

解法：每個 Sub-workflow 開頭必須標明：

```
> **調用 Skill**：`[skill-name]`  
> **使用的模板**：[Skill 中的具體模板名稱]  
> **品質標準**：[Skill 中的具體標準]
```

陷阱 4：檢查點不可量化

問題：檢查點寫「 內容完整」——AI 不知道什麼叫「完整」。

解法：檢查點必須是客觀可驗證的：

-  - [] 形狀表完整
-  - [] 形狀表每個等級都有：名稱、邊數、半徑、顏色、得分、設計意圖
-  - [] 物理參數分為「引擎全域」和「形狀 Body」兩組
-  - [] 每個 Overlay 有 z-index + 觸發時機 + 關閉方式